



HE1025 Operativsystem

Ö4: SD Kort & Direct Memory Access (DMA)



Kursen...

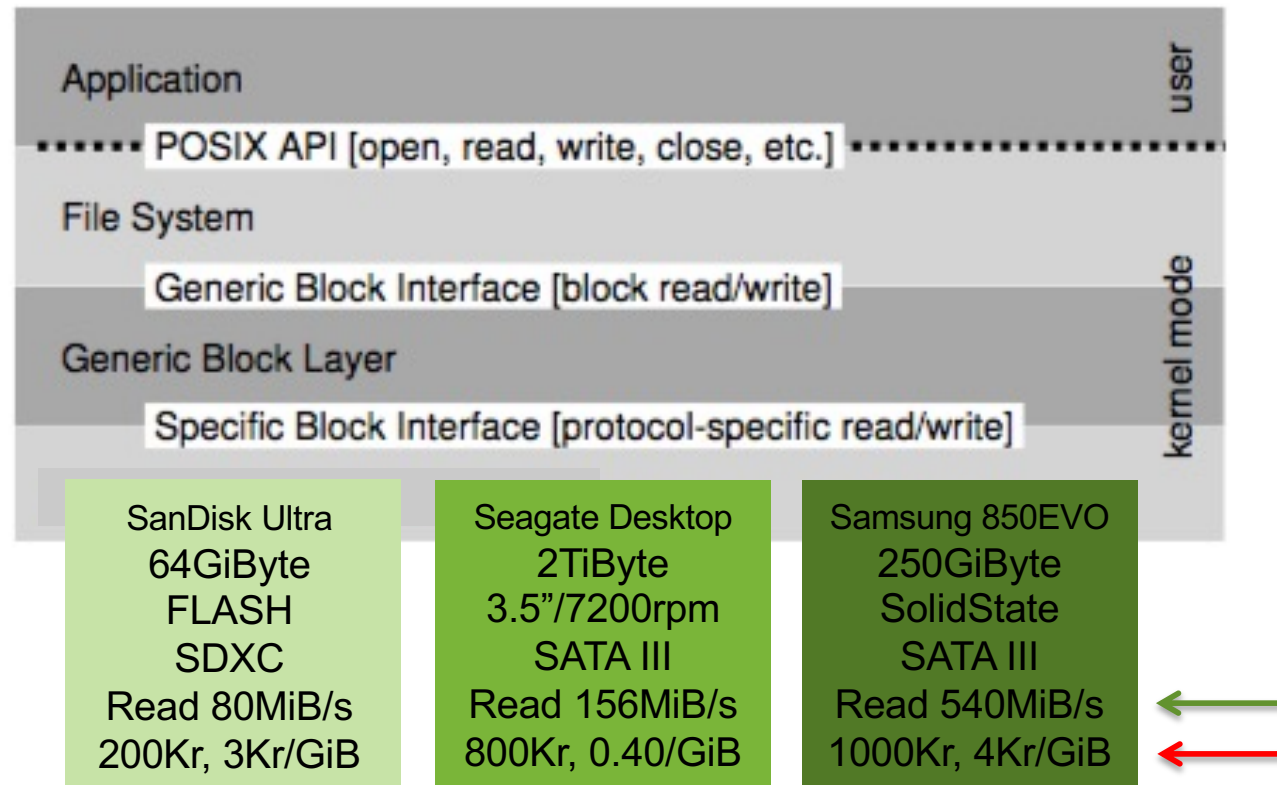
F1: V#1 Processorn.	Kap (1) 2..4 & 6
Ö1: V#2 c Pekare...	Kap 14 & c-boken
F2: V#3 Schemaläggning	Kap 7..9,(10-11), RTOS
L1: V#4 FreeRTOS	Lab #1 Handledning
F3: V#5 Segmentation	Kap 12..13, 15..17
Ö2: V#6 Buffer	t.b.d
F4: V#7 Paging	Kap 18-22, (23-24)
L2: V#8 Paging	Lab #2 Handledning
F5: C#1 Locks & CV	Kap (25) 26..27 & 28..29.1
Ö3: LINUX	Kap 5 (39)
F6: C#3 Semaphores	Kap 30..33, (34)
L3: C#4 Barberare	Lab #3 Handledning
F7: P#1 Storage	Kap (35,) 36, 40, 42 (37..39, 41 & 43..46)
Ö4: P#2 SD Card/MMA	Informativt
F8: P#3 Communication	Kap (47, 39) & 48, (49..51)
L4: P#4 Client/Server	Lab #4 Handledning

TEN1: 4+4p/4:e-del, minst 4p per 4:e-del för E!

LAB1: Närvaro L1..4



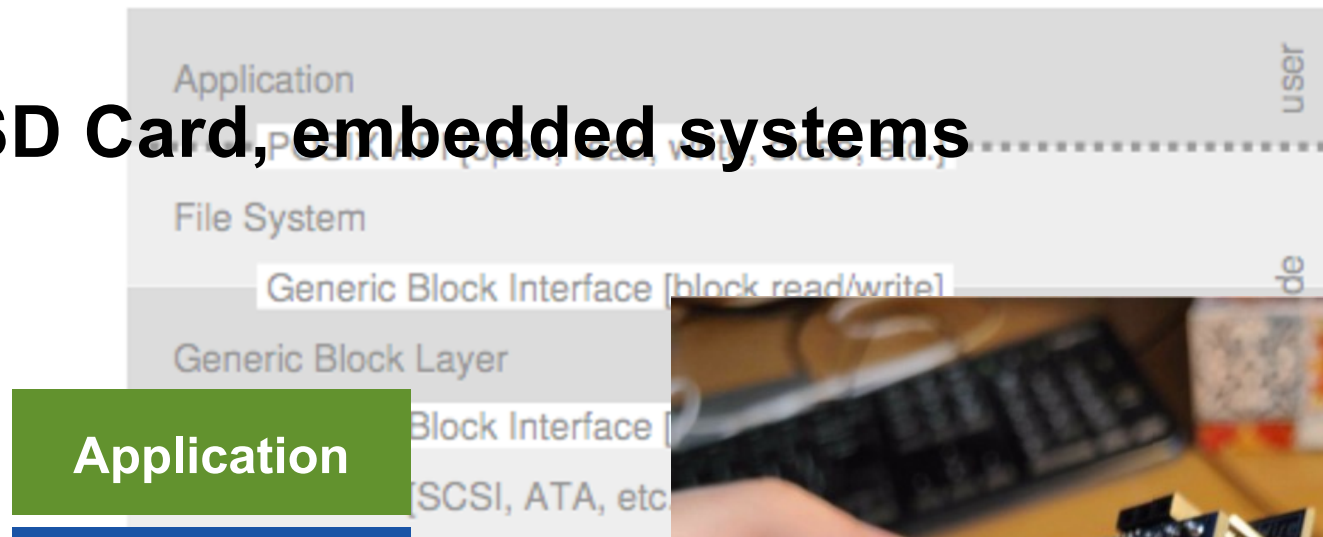
F7 Storage: Olika men samma api, DMA viktigt!





SD Card, embedded systems

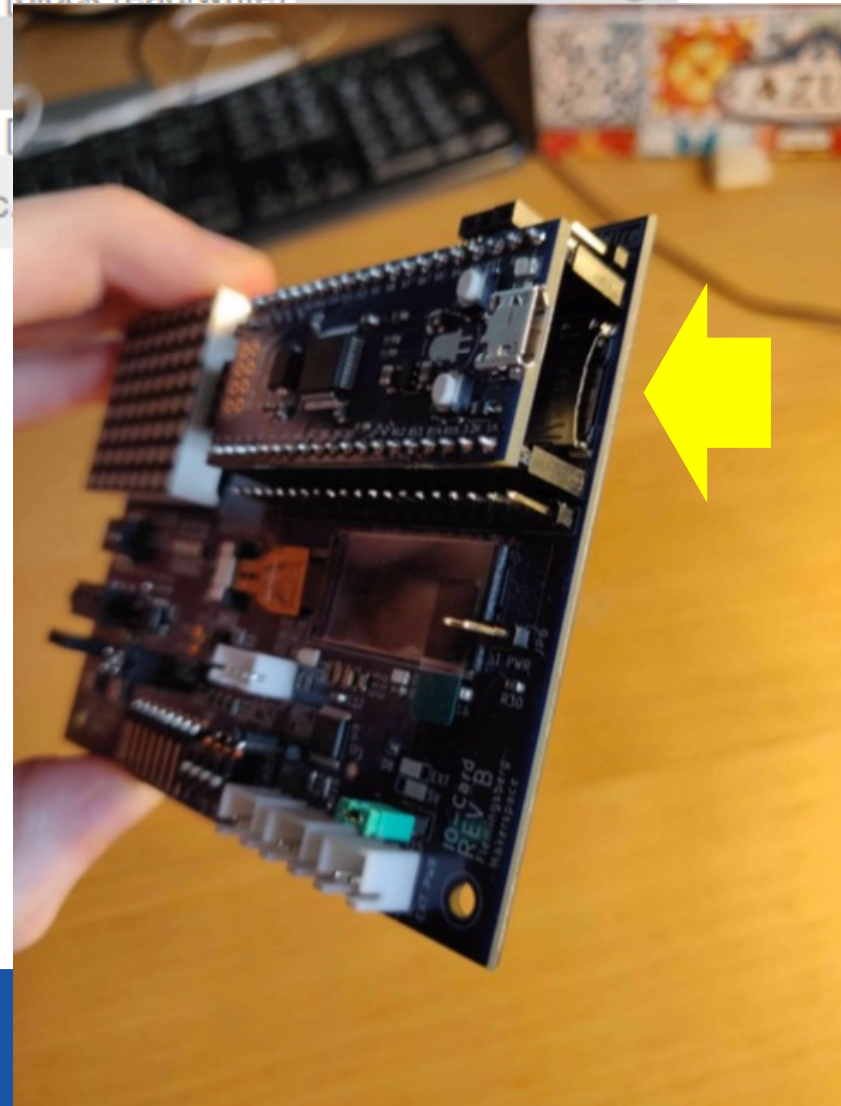
Acknowledgement research:
Linus Remahl & Olof Forsberg



Application

FatFs

MAC





FatFs – Generic FAT Filesystem Module

Application

FatFs

MAC

Features

DOS/Windows compatible FAT/exFAT filesystem.

Platform independent. Easy to port.

Very small footprint for program code and work area.

Various configuration options to support for:

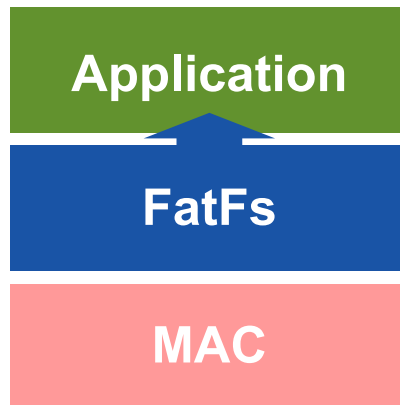
- Long file name in ANSI/OEM or Unicode.
- exFAT filesystem, 64-bit LBA and GPT.
- Thread safe for RTOS.
- Multiple volumes. (physical drives and partitions)
- Variable sector size.
- Multiple code pages including DBCS.
- Read-only, optional APIs, I/O buffer and etc...



- FAT = File Allocation Table, Windows unikt



Application interface...



File Access

- `f_open` - Open/Create a file
- `f_close` - Close an open file
- `f_read` - Read data from the file
- `f_write` - Write data to the file
- `f_lseek` - Move read/write pointer, Expand size
- `f_truncate` - Truncate file size
- `f_sync` - Flush cached data
- `f_forward` - Forward data to the stream
- `f_expand` - Allocate a contiguous block to the file
- `f_gets` - Read a string
- `f_putc` - Write a character
- `f_puts` - Write a string
- `f_printf` - Write a formatted string
- `f_tell` - Get current read/write pointer
- `f_eof` - Test for end-of-file
- `f_size` - Get size
- `f_error` - Test for an error

Directory Access

- `f_opendir` - Open a directory
- `f_closedir` - Close an open directory
- `f_readdir` - Read a directory item
- `f_findfirst` - Open a directory and read the first item matched
- `f_findnext` - Read a next item matched

File and Directory Management

- `f_stat` - Check existence of a file or sub-directory
- `f_unlink` - Remove a file or sub-directory
- `f_rename` - Rename/Move a file or sub-directory
- `f_chmod` - Change attribute of a file or sub-directory
- `f_ftime` - Change timestamp of a file or sub-directory
- `f_mkdir` - Create a sub-directory
- `f_chdir` - Change current directory
- `f_chdrive` - Change current drive
- `f_getcwd` - Retrieve the current directory and drive

Volume Management and System Configuration

- `f_mount` - Register/Unregister the work area of the volume
- `f_mkfs` - Create an FAT volume on the logical drive
- `f_fdisk` - Create partitions on the physical drive
- `f_getfree` - Get free space on the volume
- `f_getlabel` - Get volume label
- `f_setlabel` - Set volume label
- `f_setcp` - Set active code page



```
FATFS fs;           /* Handle for the mounted filesystem */
volatile FRESULT fr; /* FatFs return code */
FIL fil;           /* File handle */
UINT bw = 999;     /* Used for bytes written, and bytes read */
char buf[128] = "write this to a file\n"; /* Text to write to file */

/* Sets a valid date for when writing to file */
set_fattime(1980, 1, 1, 0, 0, 0);

/* This function "mounts" the SD-card, thus makes it available */
fr = f_mount(&fs, "", 1);

/* This function opens a file that could be written to */
fr = f_open(&fil, "file.txt", FA_WRITE);

/* Write some text to the file */
for(int i = 0; i < 10; i++)
    fr = f_write(&fil, buf, strlen(buf), &bw);

/* Make sure writes are committed to the file */
delay_1ms(100); f_sync(&fil);

/* Close the file */
delay_1ms(100); f_close(&fil);

/* Unmount file system! */
f_unmount("");
```

≈2ms

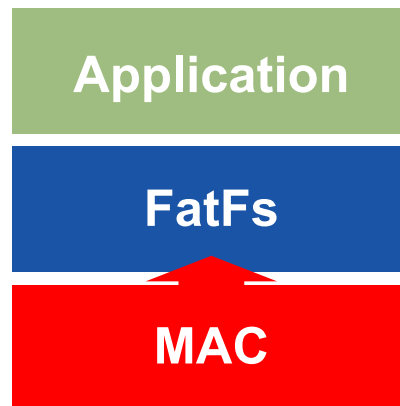
≈1μs (nb)

≈1ms

≈1ms

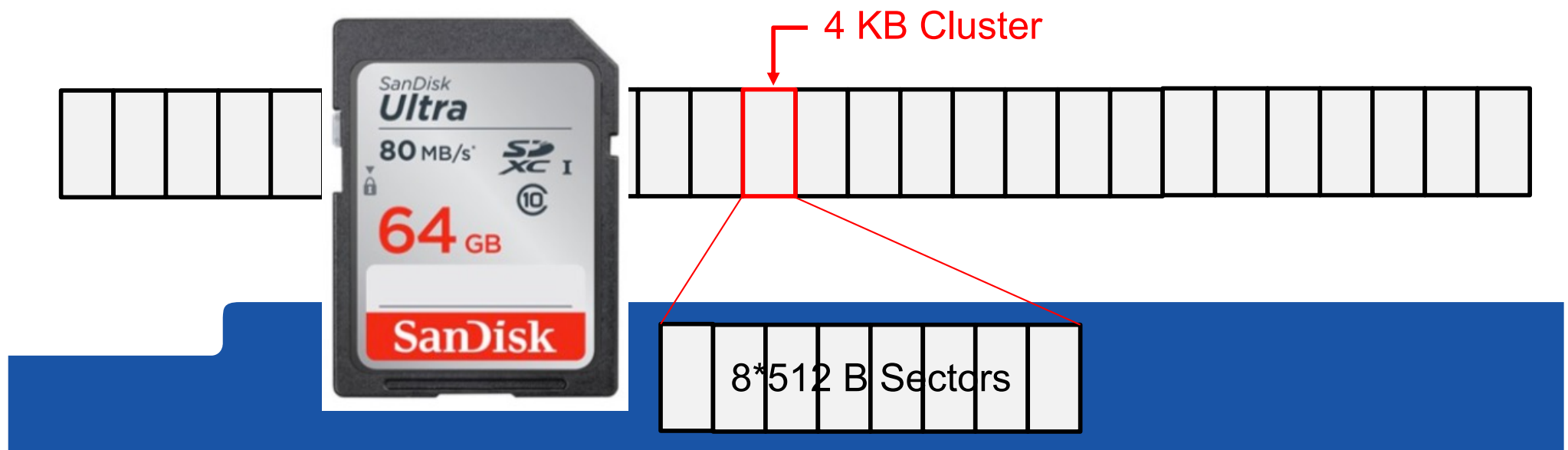


Media interface...



Storage Device Controls

- disk_status - Get device status
- disk_initialize - Initialize device
- disk_read - Read data
- disk_write - Write data
- disk_ioctl - Control device dependent functions





Media interface...

Volume Boot Record (VBR)

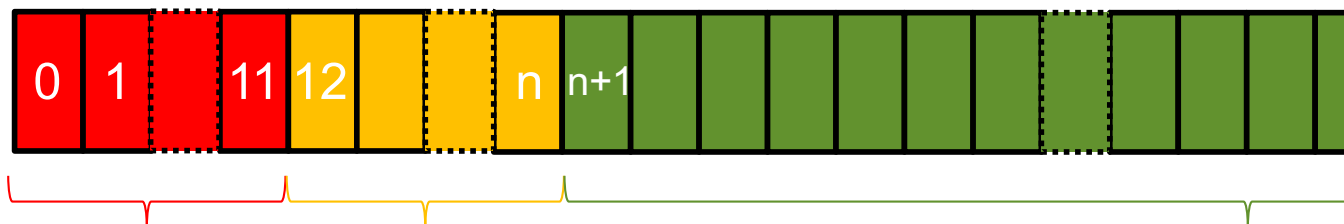
- BIOS Parameter Block (BPB)
- OEM Parameters
- Checksum sector (11)

File Allocation Table (FAT)

- Ett word för varje cluster i data area

Data Area

- Resten av clustren i disken, minst 65526 st



VBR

FAT

DATA AREA

(Minst 16 cluster)

Field name	Offset	Size	Field name	Offset	Size
BS_JmpBoot	0	3	BPB_FATs32	36	4
			BPB_ExtFlags	40	2
BS_OEMName	3	8	BPB_FSVer	42	2
BPB_BytsPerSec	11	2	BPB_RootClus	44	4
BPB_SecPerClus	13	1	BPB_FSInfo	48	2
			BPB_BkBootSec	50	2
BPB_RsvdSecCnt	14	2	BPB_Reserved	52	12
			BS_DrvNum	64	1
BPB_NumFATs	16	1	BS_Reserved	65	1
			BS_BootSig	66	1
			BS_VolID	67	4
			BS_VolLab	71	11
			BS_FilSysType	82	8
			BS_BootCode32	90	420
			BS_BootSign	510	2
BPB_RootEntCnt	17	2			
BPB_TotSec16	19	2			
BPB_Media	21	1			
BPB_FATs16	22	2			
BPB_SecPerTrk	24	2			
BPB_NumHeads	26	2			
BPB_HiddSec	28	4			
BPB_TotSec32	32	4			



Media interface...

Offset	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
+0000	F0	FF ⁰⁰	FF	0F	FF	FF ⁰¹	FF	0F	FF	FF ⁰²	FF	0F	04	00 ⁰³	00	00
+0010	05	00 ⁰⁴	00	00	06	00 ⁰⁵	00	00	07	00 ⁰⁶	00	00	08	00 ⁰⁷	00	00
+0020	FF	FF ⁰⁸	FF	0F	0A	00 ⁰⁹	00	00	14	00 ^{0A}	00	00	0C	00 ^{0B}	00	00
+0030	0D	00 ^{0C}	00	00	0E	00 ^{0D}	00	00	0F	00 ^{0E}	00	00	10	00 ^{0F}	00	00
+0040	11	00 ¹⁰	00	00	FF	FF ¹¹	FF	0F	00	00 ¹²	00	00	FF	FF ¹³	FF	0F
+0050	15	00 ¹⁴	00	00	16	00 ¹⁵	00	00	19	00 ¹⁶	00	00	F7	FF ¹⁷	FF	0F
+0060	F7	FF ¹⁸	FF	0F	1A	00 ¹⁹	00	00	FF	FF ^{1A}	FF	0F	00	00 ^{1B}	00	00
+0070	00	00 ^{1C}	00	00	F7	FF ^{1D}	FF	0F	00	00 ^{1E}	00	00	00	00 ^{1F}	00	00



VBR

FAT

DATA

(Minst 16 cluster)

Field name	Offset	Size
BPB_FATs32	36	4
BPB_ExtFlags	40	2
BPB_FSVer	42	2
BPB_RootClus	44	4
BPB_FSInfo	48	2
BPB_BkBootSec	50	2
BPB_Reserved	52	12
BS_DrvNum	64	1
BS_Reserved	65	1
BS_BootSig	66	1
BS_VolID	67	4
BS_VolLab	71	11
BS_FilSysType	82	8
BS_BootCode32	90	420
BS_BootSign	510	2

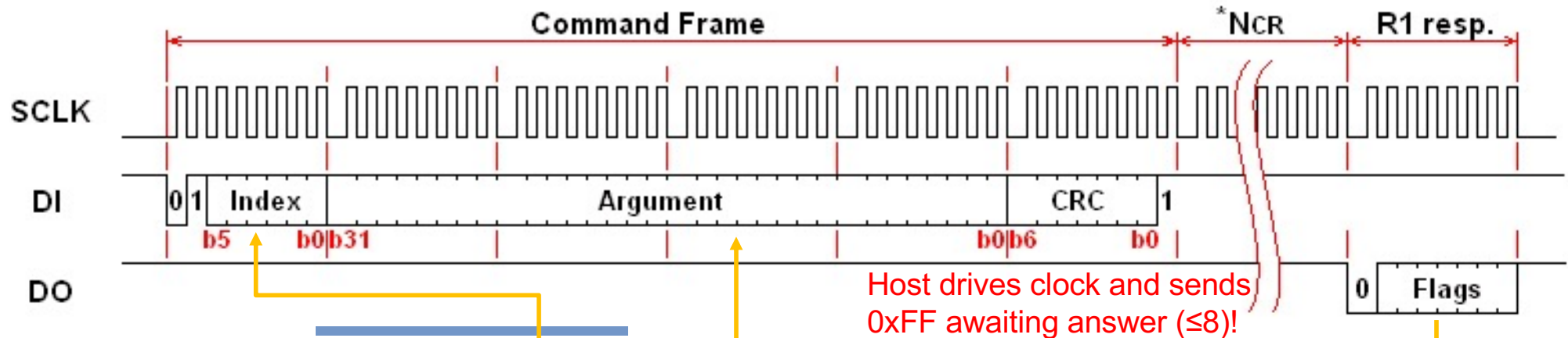
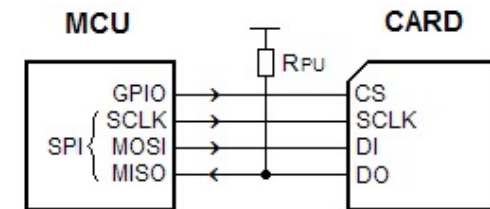
2

Dir[0] 32 Byte Entry .
Dir[1] .. (/.. Not if root)
Dir[2] 3 foo.txt metadata
Dir[3] 9 log.txt metadata
Dir[4] B moo.txt metadata
Dir[5] 13 usr.dir metadata

FAT32	Meaninig
0x00000000	Free
0x00000001	Reserved
0x00000002 - 0xFFFFFFFF6	In use (value is link to next)
0xFFFFFFFF7	Bad cluster
0xFFFFFFFF8 - 0xFFFFFFFFF	In use (end of chain)

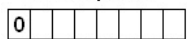


Physical interface (SPI)...



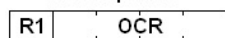
MAC

R1 Response



- In Idle State
- Erase Reset
- Illegal Command
- Command CRC Error
- Erase Sequence Error
- Address Error
- Parameter Error

R3 Response



- OCR (32bit)
- Same as R1



Command Index	Argument	Response	Data	Abbreviation	Description
CMD0	None(0)	R1	No	GO_IDLE_STATE	Software reset.
CMD1	None(0)	R1	No	SEND_OP_COND	Initiate initialization process.
ACMD41(*1)	*2	R1	No	APP_SEND_OP_COND	For only SDC. Initiate initialization process.
CMD8	*3	R7	No	SEND_IF_COND	For only SDC V2. Check voltage range.
CMD9	None(0)	R1	Yes	SEND_CSD	Read CSD register.
CMD10	None(0)	R1	Yes	SEND_CID	Read CID register.
CMD12	None(0)	R1b	No	STOP_TRANSMISSION	Stop to read data.
CMD16	Block length[31:0]	R1	No	SET_BLOCKLEN	Change R/W block size.
CMD17	Address[31:0]	R1	Yes	READ_SINGLE_BLOCK	Read a block.
CMD18	Address[31:0]	R1	Yes	READ_MULTIPLE_BLOCK	Read multiple blocks.
CMD23	Number of blocks[15:0]	R1	No	SET_BLOCK_COUNT	For only MMC. Define number of blocks to transfer with next multi-block read/write command.
ACMD23(*1)	Number of blocks[22:0]	R1	No	SET_WR_BLOCK_ERASE_COUNT	For only SDC. Define number of blocks to pre-erase with next multi-block write command.
CMD24	Address[31:0]	R1	Yes	WRITE_BLOCK	Write a block.
CMD25	Address[31:0]	R1	Yes	WRITE_MULTIPLE_BLOCK	Write multiple blocks.
CMD55(*1)	None(0)	R1	No	APP_CMD	Leading command of ACMD<n> command.
CMD58	None(0)	R3	No	READ_OCR	Read OCR.

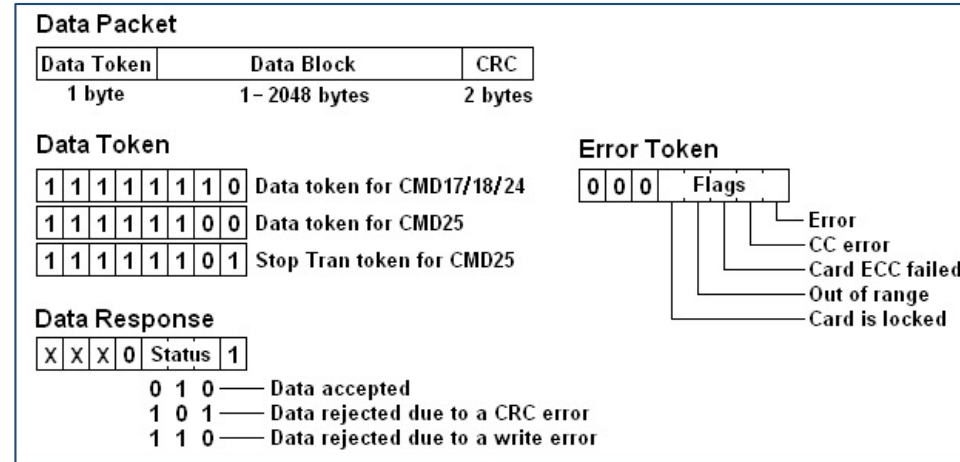
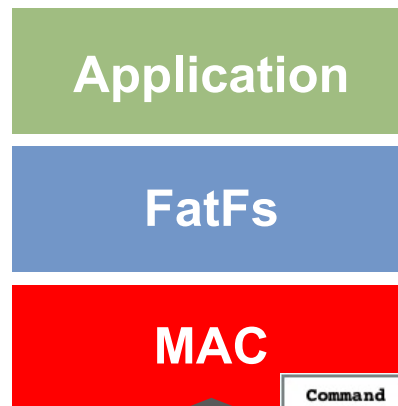
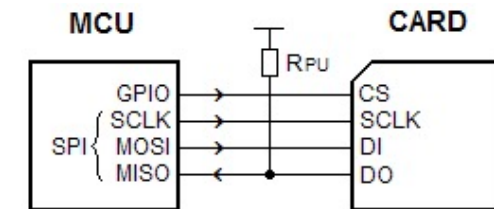
*1:ACMD<n> means a command sequence of CMD55-CMD<n>.

*2: Rsv(0)[31], HCS[30], Rsv(0)[29:0]

*3: Rsv(0)[31:12], Supply Voltage(1)[11:8], Check Pattern(0xAA)[7:0]



Physical interface (SPI)...

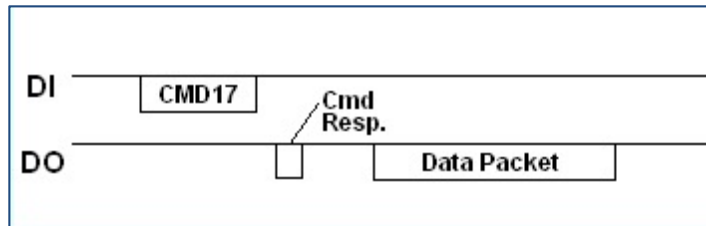
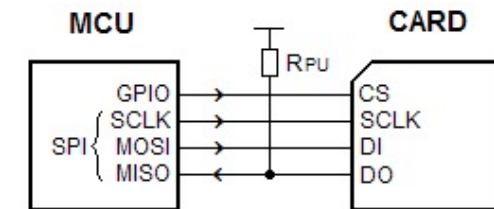


Command Index	Argument	Response	Data	Abbreviation	Description
CMD0	None(0)	R1	No	GO_IDLE_STATE	Software reset.
CMD1	None(0)	R1	No	SEND_OP_COND	Initiate initialization process.
ACMD41(*1)	*2	R1	No	APP_SEND_OP_COND	For only SDC. Initiate initialization process.
CMD8	*3	R7	No	SEND_IF_COND	For only SDC V2. Check voltage range.
CMD9	None(0)	R1	Yes	SEND_CSD	Read CSD register.
CMD10	None(0)	R1	Yes	SEND_CID	Read CID register.
CMD12	None(0)	R1b	No	STOP_TRANSMISSION	Stop to read data.
CMD16	Block length[31:0]	R1	No	SET_BLOCKLEN	Change R/W block size.
CMD17	Address[31:0]	R1	Yes	READ_SINGLE_BLOCK	Read a block.
CMD18	Address[31:0]	R1	Yes	READ_MULTIPLE_BLOCK	Read multiple blocks.
CMD23	Number of blocks[15:0]	R1	No	SET_BLOCK_COUNT	For only MMC. Define number of blocks to transfer with next multi-block read/write command.
ACMD23(*1)	Number of blocks[22:0]	R1	No	SET_WR_BLOCK_ERASE_COUNT	For only SDC. Define number of blocks to pre-erase with next multi-block write command.
CMD24	Address[31:0]	R1	Yes	WRITE_BLOCK	Write a block.
CMD25	Address[31:0]	R1	Yes	WRITE_MULTIPLE_BLOCK	Write multiple blocks.
CMD55(*1)	None(0)	R1	No	APP_CMD	Leading command of ACMD<n> command.
CMD58	None(0)	R3	No	READ_OCR	Read OCR.

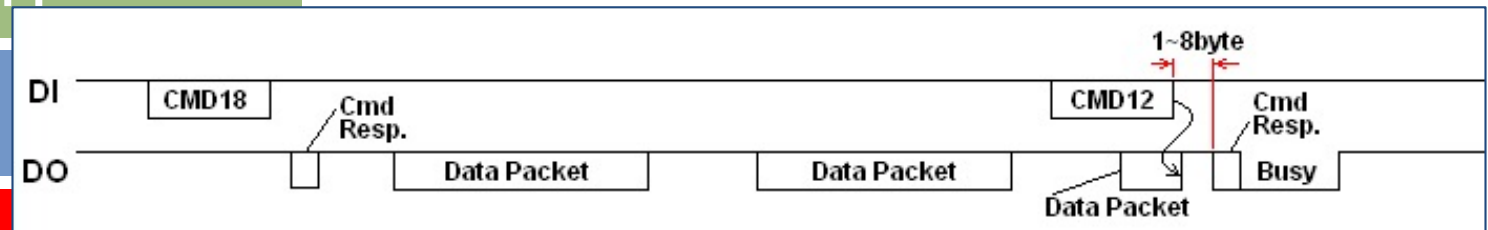
*1:ACMD<n> means a command sequence of CMD55-CMD<n>.
*2: Rsv(0)[31], HCS[30], Rsv(0)[29:0]
*3: Rsv(0)[31:12], Supply Voltage(1)[11:8], Check Pattern(0xAA)[7:0]



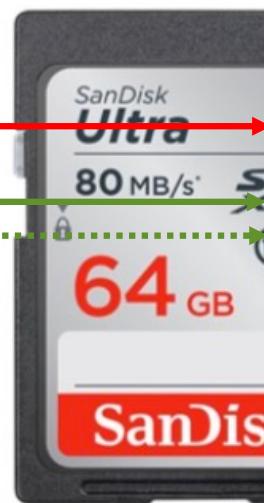
Physical interface (SPI)...



Application



MAC



Command Index	Argument	Response	Data	Abbreviation	Description
CMD0	None(0)	R1	No	GO_IDLE_STATE	Software reset.
CMD1	None(0)	R1	No	SEND_OP_COND	Initiate initialization process.
ACMD41(*1)	*2	R1	No	APP_SEND_OP_COND	For only SDC. Initiate initialization process.
CMD8	*3	R7	No	SEND_IF_COND	For only SDC V2. Check voltage range.
CMD9	None(0)	R1	Yes	SEND_CSD	Read CSD register.
CMD10	None(0)	R1	Yes	SEND_CID	Read CID register.
CMD12	None(0)	R1b	No	STOP_TRANSMISSION	Stop to read data.
CMD16	Block length[31:0]	R1	No	SET_BLOCKLEN	Change R/W block size.
CMD17	Address[31:0]	R1	Yes	READ_SINGLE_BLOCK	Read a block.
CMD18	Address[31:0]	R1	Yes	READ_MULTIPLE_BLOCK	Read multiple blocks.
CMD23	Number of blocks[15:0]	R1	No	SET_BLOCK_COUNT	For only MMC. Define number of blocks to transfer with next multi-block read/write command.
ACMD23(*1)	Number of blocks[22:0]	R1	No	SET_WR_BLOCK_ERASE_COUNT	For only SDC. Define number of blocks to pre-erase with next multi-block write command.
CMD24	Address[31:0]	R1	Yes	WRITE_BLOCK	Write a block.
CMD25	Address[31:0]	R1	Yes	WRITE_MULTIPLE_BLOCK	Write multiple blocks.
CMD55(*1)	None(0)	R1	No	APP_CMD	Leading command of ACMD<n> command.
CMD58	None(0)	R3	No	READ_OCR	Read OCR.

*1:ACMD<n> means a command sequence of CMD55-CMD<n>.

*2: Rsv(0)[31], HCS[30], Rsv(0)[29:0]

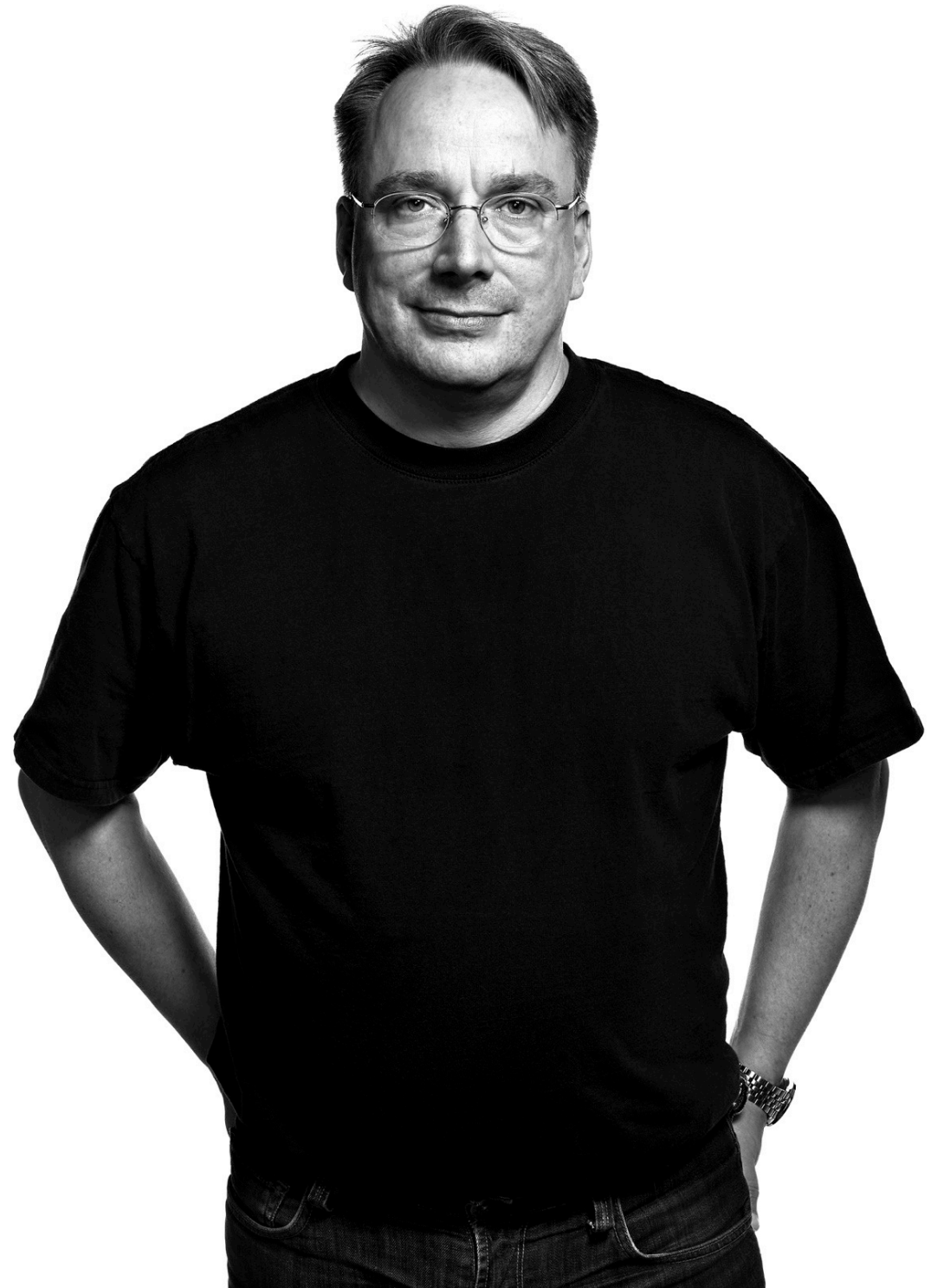
*3: Rsv(0)[31:12], Supply Voltage(1)[11:8], Check Pattern(0xAA)[7:0]

Att läsa ett (1) cluster (om 4kB), även om det finns hårdvarustöd på Byte-nivå, är krävande...



Mindmap

SD Card
DMA





Historik

210510 0.5 AC Draft